

MCQ Questions(Only for Practice)

Q1.

```
public class Test {  
  
    static {  
        System.out.println(x);  
    }  
  
    static int x = 10;  
  
    public static void main(String[] args) {  
  
    }  
}
```

What will happen?

- A. 0
- B. 10
- C. Compilation Error
- D. Runtime Exception

Q2.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int i = 1;  
  
        i = i++ + ++i + i++;  
  
        System.out.println(i);  
    }  
}
```

```
}
```

Output?

- A. 5
- B. 6
- C. 7
- D. 8

Q3.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        byte b = 10;  
  
        b = b * 5;  
  
        System.out.println(b);  
    }  
}
```

Result?

- A. 50
- B. Compilation Error
- C. Runtime Error
- D. Garbage Value

Q4.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int x = 10;  
  
    }  
}
```

```
        System.out.println(x+++x);
    }
}
```

Output?

- A. 20
- B. 21
- C. Compilation Error
- D. Runtime Error

Q5.

```
public class Test {

    public static void main(String[] args) {

        final int x = 10;

        switch(5) {

            case x:
                System.out.println("A");
        }
    }
}
```

Output?

- A. A
- B. Compilation Error
- C. Runtime Error
- D. No Output

Q6.

Which statement is TRUE regarding command line arguments?

- A. They are stored in JVM heap memory only
- B. They are always integers
- C. They are stored as a String array in `main()`
- D. JVM converts them automatically into primitive types

Q7.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int x = 0;  
  
        for(System.out.print("A"); x <  
3;System.out.print("B")) {  
            x++;  
        }  
    }  
}
```

Output?

- A. AB BB
- B. AB BB
- C. AB BB
- D. AB BB with newline

Q8.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int[] arr = new int[2];
```

```
        arr[0] = 10;
        arr[1] = 20;

        System.out.println(arr[-1]);
    }
}
```

Result?

- A. 10
- B. 20
- C. ArrayIndexOutOfBoundsException
- D. Compilation Error

Q9.

```
public class Test {

    public static void main(String[] args) {

        int[][] arr = new int[2][];

        arr[0] = new int[2];

        System.out.println(arr[1][0]);
    }
}
```

Output?

- A. 0
- B. NullPointerException
- C. ArrayIndexOutOfBoundsException
- D. Compilation Error

Q10.

```
public class Test {
```

```

public static void main(String[] args) {

    int[] arr = {1,2,3};

    System.out.println(arr instanceof Object);
}
}

```

Output?

- A. true
- B. false
- C. Compilation Error
- D. Runtime Error

Q11.

Which of the following is NOT stored in Stack Memory?

- A. Local Variables
- B. Method Calls
- C. Object Instances
- D. References of Objects

Q12.

```

public class Test {

    public static void main(String[] args) {

        int x = 10;

        do {
            System.out.print(x + " ");
            x--;
        } while(x > 10);
    }
}

```

```
    }  
}
```

Output?

- A. 10
- B. 10 9
- C. Infinite Loop
- D. No Output

Q13.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int[][] arr = {  
            {1,2},  
            {3,4,5},  
            {6}  
        };  
  
        System.out.println(arr.length);  
        System.out.println(arr[1].length);  
    }  
}
```

Output?

- A.
2
3
- B.

3

2

C.

3

3

D. Compilation Error

Q14.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int x = 5;  
  
        while(x-- > 0) {  
            System.out.print(x + " ");  
        }  
    }  
}
```

Output?

A.

5 4 3 2 1

B.

4 3 2 1 0

C.

5 4 3 2 1 0

D. Infinite Loop

Q15.

Which statement about Java primitive data types is correct?

- A. `char` is signed
- B. `float` is 64-bit
- C. `boolean` size is JVM-dependent
- D. `long` is 32-bit

Q16.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int arr[] = new int[3];  
  
        System.out.println(arr.getClass().getName());  
    }  
}
```

Output?

- A. `int[]`
- B. `[]`
- C. `java.lang.Array`
- D. Compilation Error

Q17.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int x = 10;  
  
        if(x > 5)
```

```
        if(x > 8)
            System.out.println("A");
        else
            System.out.println("B");
    }
}
```

Output?

- A. A
- B. B
- C. No Output
- D. Compilation Error

Q18.

```
public class Test {

    public static void main(String[] args) {

        int[][] arr = new int[2][2];

        System.out.println(arr[0]);
    }
}
```

Output?

- A. 0
- B. null
- C. Memory Address-like representation
- D. Compilation Error

Q19.

Which of the following is TRUE about Java arrays?

- A. Arrays are dynamically resized automatically
- B. Arrays are objects in Java
- C. Arrays can store different primitive types together
- D. Array size can be changed after creation

Q20.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        int x = 1;  
  
        switch(x) {  
  
            case 0:  
                System.out.println("Zero");  
                break;  
  
            default:  
                System.out.println("Default");  
  
            case 1:  
                System.out.println("One");  
        }  
    }  
}
```

Output?

A.

One

B.

Default

One

C.

Zero

D. Compilation Error

Q21.

```
class Test {  
  
    int x = 10;  
  
    Test() {  
        this(20);  
        System.out.println("Default");  
    }  
  
    Test(int x) {  
        this.x = x;  
    }  
  
    public static void main(String[] args) {  
        Test t = new Test();  
        System.out.println(t.x);  
    }  
}
```

Output?

A.

Default

10

B.

20

Default

C.

Default

20

D. Compilation Error

Q22.

```
class Parent {  
  
    static void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    static void show() {  
        System.out.println("Child");  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Parent p = new Child();  
        p.show();  
    }  
}
```

Output?

A. Parent

B. Child

C. Compilation Error

D. Runtime Error

Q23.

Which statement about constructor overloading is TRUE?

- A. Constructors can differ only by return type
- B. Constructors can be inherited
- C. Multiple constructors with different parameter lists are allowed
- D. Constructors can be overridden

Q24.

```
class Parent {  
  
    Parent() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    Child() {  
        System.out.println("Child");  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
        Child c = new Child();  
    }  
}
```

Output?

A.

Child
Parent

B.

Parent

Child

C. Compilation Error

D. Runtime Error

Q25.

```
class Parent {  
  
    int x = 10;  
}  
  
class Child extends Parent {  
  
    int x = 20;  
  
    void print() {  
        System.out.println(super.x);  
        System.out.println(this.x);  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
        new Child().print();  
    }  
}
```

Output?

A.

20

20

B.

10

20

C.

10

10

D. Compilation Error

Q26.

```
class Test {  
  
    static int count = 0;  
  
    Test() {  
        count++;  
    }  
  
    public static void main(String[] args) {  
  
        new Test();  
        new Test();  
        new Test();  
  
        System.out.println(Test.count);  
    }  
}
```

Output?

A. 0

B. 1

C. 2

D. 3

Q27.

Which statement about the **final** keyword is correct?

- A. final class can be inherited
- B. final method can be overridden
- C. final variable cannot be reassigned
- D. final constructor is allowed

Q28.

```
class Parent {  
  
    void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    void show() {  
        System.out.println("Child");  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Parent p = new Child();  
        p.show();  
    }  
}
```

Output?

- A. Parent
- B. Child

- C. Compilation Error
- D. Runtime Error

Q29.

```
class Test {  
  
    int x;  
  
    Test() {  
        this(10);  
    }  
  
    Test(int x) {  
        this.x = x;  
    }  
  
    public static void main(String[] args) {  
  
        Test t = new Test();  
  
        System.out.println(t.x);  
    }  
}
```

Output?

- A. 0
- B. 10
- C. Garbage Value
- D. Compilation Error

Q30.

```
abstract class Demo {  
  
    abstract void show();  
}
```

```

        Demo() {
            System.out.println("Constructor");
        }
    }

    class Test extends Demo {

        void show() {
            System.out.println("Show");
        }

        public static void main(String[] args) {
            new Test().show();
        }
    }

```

Output?

A.

Show
Constructor

B.

Constructor
Show

C. Compilation Error

D. Runtime Error

Q31.

Which of the following is TRUE about interfaces in Java?

A. Interface methods are private by default

B. Interface can contain abstract methods

- C. Interface objects can be created directly
- D. Interface supports constructors

Q32.

```
interface A {  
  
    int x = 10;  
}  
  
public class Test implements A {  
  
    public static void main(String[] args) {  
  
        x = 20;  
  
        System.out.println(x);  
    }  
}
```

Result?

- A. 20
- B. 10
- C. Compilation Error
- D. Runtime Error

Q33.

```
class Parent {  
  
    Parent() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {
```

```

        Child() {
            super();
            System.out.println("Child");
        }
    }

    public class Test {

        public static void main(String[] args) {
            new Child();
        }
    }

```

Output?

A.

Parent
Child

B.

Child
Parent

C. Compilation Error
D. Runtime Error

Q34.

```

class Test {

    static {
        System.out.println("Static Block");
    }

    {
        System.out.println("Instance Block");
    }
}

```

```

    }

    Test() {
        System.out.println("Constructor");
    }

    public static void main(String[] args) {
        new Test();
    }
}

```

Output?

A.

Constructor
Instance Block
Static Block

B.

Static Block
Instance Block
Constructor

C.

Instance Block
Static Block
Constructor

D. Compilation Error

Q35.

Which statement is TRUE regarding method overriding?

- A. Private methods can be overridden
- B. Static methods participate in runtime polymorphism
- C. Overridden method cannot reduce visibility
- D. Constructors can be overridden

Q36.

```
class Parent {  
  
    void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
  
    void display() {  
        System.out.println("Child");  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Parent p = new Child();  
  
        p.display();  
    }  
}
```

Result?

- A. Parent
- B. Child
- C. Compilation Error
- D. Runtime Error

Q37.

```
interface A {
```

```

        void show();
    }

    interface B {
        void show();
    }

    class Test implements A, B {

        public void show() {
            System.out.println("Hello");
        }

        public static void main(String[] args) {
            new Test().show();
        }
    }

```

Output?

- A. Hello
- B. Compilation Error
- C. Runtime Error
- D. Ambiguous Method Error

Q38.

```

class Test {

    static int x = 10;

    static void change() {
        x = 20;
    }

    public static void main(String[] args) {

```



```
        Test.change();

        System.out.println(Test.x);
    }
}
```

Output?

- A. 10
- B. 20
- C. Compilation Error
- D. Runtime Error

Q39.

Which of the following demonstrates runtime polymorphism?

- A. Method Overloading
- B. Constructor Overloading
- C. Method Overriding
- D. Static Method Hiding

Q40.

```
abstract class A {

    abstract void show();
}

class B extends A {

}

public class Test {

    public static void main(String[] args) {
```

```
        B obj = new B();
    }
}
```

Result?

- A. Object Created Successfully
- B. Runtime Error
- C. Compilation Error
- D. NullPointerException

Q41.

```
class MyThread extends Thread {

    public void run() {
        System.out.println("Thread");
    }
}

public class Test {

    public static void main(String[] args) {

        MyThread t = new MyThread();

        t.start();
        t.start();
    }
}
```

Result?

- A. Thread printed twice
- B. Compilation Error
- C. IllegalStateException
- D. No Output

Q42.

```
class MyThread extends Thread {  
  
    public void run() {  
  
        for(int i=0; i<3; i++) {  
            System.out.print(i + " ");  
        }  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) throws  
Exception {  
  
        MyThread t = new MyThread();  
  
        t.start();  
  
        t.join();  
  
        System.out.println("Main");  
    }  
}
```

Output?

A.

Main
0 1 2

B.

0 1 2 Main

- C. Compilation Error
- D. Output unpredictable

Q43.

Which method checks whether a thread is still running?

- A. run()
- B. sleep()
- C. isAlive()
- D. join()

Q44.

```
class MyThread implements Runnable {  
  
    public void run() {  
  
        System.out.println(Thread.currentThread().getName());  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Thread t = new Thread(new MyThread());  
  
        t.start();  
    }  
}
```

Output?

- A. main
- B. Thread-0

- C. Compilation Error
- D. Runtime Error

Q45.

```
class Counter {  
  
    int count = 0;  
  
    synchronized void increment() {  
        count++;  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Counter c = new Counter();  
  
        c.increment();  
  
        System.out.println(c.count);  
    }  
}
```

Output?

- A. 0
- B. 1
- C. Compilation Error
- D. Runtime Error

Q46.

Which statement about synchronization is TRUE?

- A. Synchronization increases thread interference
- B. Synchronization prevents race conditions
- C. Synchronization works only with static methods
- D. Synchronization stops thread execution permanently

Q47.

```
class MyThread extends Thread {  
  
    public void run() {  
  
        System.out.println(Thread.currentThread().getPriority());  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        MyThread t = new MyThread();  
  
        t.setPriority(Thread.MAX_PRIORITY);  
  
        t.start();  
    }  
}
```

Output?

- A. 1
- B. 5
- C. 10
- D. Compilation Error

Q48.

```
import java.util.*;
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        ArrayList<Integer> list = new ArrayList<>();  
  
        list.add(10);  
        list.add(20);  
  
        list.add(1, 50);  
  
        System.out.println(list);  
    }  
}
```

Output?

A.

[10, 20, 50]

B.

[10, 50, 20]

C.

[50, 10, 20]

D. Compilation Error

Q49.

Which collection does NOT allow duplicate elements?

A. ArrayList

B. LinkedList

C. HashSet

D. Vector

Q50.

```
import java.util.*;

public class Test {

    public static void main(String[] args) {

        HashSet<String> set = new HashSet<>();

        set.add("Java");
        set.add("Java");

        System.out.println(set.size());
    }
}
```

Output?

- A. 0
- B. 1
- C. 2
- D. Compilation Error

Q51.

```
import java.util.*;

public class Test {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1,2,3);

        list.set(1,100);
    }
}
```



```
        System.out.println(list);
    }
}
```

Output?

A.

[1, 2, 3]

B.

[1, 100, 3]

C. UnsupportedOperationException

D. Compilation Error

Q52.

```
import java.util.*;

public class Test {

    public static void main(String[] args) {

        PriorityQueue<Integer> pq =
            new PriorityQueue<>();

        pq.add(30);
        pq.add(10);
        pq.add(20);

        System.out.println(pq.poll());
    }
}
```

Output?

- A. 30
- B. 20
- C. 10
- D. Random Value

Q53.

Which interface is implemented to define natural sorting order?

- A. Comparator
- B. Comparable
- C. Collection
- D. Cloneable

Q54.

```
class Box<T> {  
  
    T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
  
    T get() {  
        return value;  
    }  
}  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Box<String> b = new Box<>("Java");  
  
        System.out.println(b.get());  
    }  
}
```

```
}
```

Output?

- A. Java
- B. null
- C. Compilation Error
- D. Runtime Error

Q55.

```
import java.util.*;

public class Test {

    public static void main(String[] args) {

        Map<Integer,String> map =
            new HashMap<>();

        map.put(1, "Java");
        map.put(1, "Python");

        System.out.println(map.get(1));
    }
}
```

Output?

- A. Java
- B. Python
- C. null
- D. Compilation Error

Q56.

```
import java.util.*;
```

```

public class Test {

    public static void main(String[] args) {

        Stack<Integer> stack = new Stack<>();

        stack.push(10);
        stack.push(20);

        System.out.println(stack.pop());
    }
}

```

Output?

- A. 10
- B. 20
- C. null
- D. Compilation Error

Q57.

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("A");
        list.add("B");

        for(String s : list) {

            list.remove(s);

```

```

        }
    }
}

```

Result?

- A. No Error
- B. ConcurrentModificationException
- C. Compilation Error
- D. NullPointerException

Q58.

Which wildcard provides read-only access in Generics?

- A. <? super T>
- B. <? extends T>
- C. <? implements T>
- D. <? type T>

Q59.

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        TreeSet<Integer> set =
            new TreeSet<>();

        set.add(50);
        set.add(10);
        set.add(30);

        System.out.println(set);
    }
}

```

```
}
```

Output?

A.

[50, 10, 30]

B.

[10, 30, 50]

C.

[30, 10, 50]

D. Compilation Error

Q60.

```
class Test<T extends Number> {  
  
    T value;  
  
    Test(T value) {  
        this.value = value;  
    }  
  
    void show() {  
        System.out.println(value);  
    }  
  
    public static void main(String[] args) {  
  
        Test<Integer> t1 =  
            new Test<>(100);  
  
        Test<String> t2 =  
            new Test<>("Java");  
    }  
}
```

```
}
```

Result?

- A. 100 Java
- B. Compilation Error
- C. Runtime Error
- D. null

Q61.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s1 = "Java";  
        String s2 = new String("Java");  
        String s3 = "Ja" + "va";  
  
        System.out.println(s1 == s2);  
        System.out.println(s1 == s3);  
        System.out.println(s1.equals(s2));  
    }  
}
```

Output?

A.

true
true
true

B.

false
true
true

C.

false

false

true

D.

true

false

true

Q62.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s = "Java";  
  
        s.concat(" Programming");  
  
        System.out.println(s);  
    }  
}
```

Output?

A. Java Programming

B. Java

C. Compilation Error

D. Runtime Error

Q63.

Which statement about **StringBuilder** is TRUE?

A. It is immutable

B. It is synchronized

C. It is mutable and non-synchronized

D. It cannot modify existing strings

Q64.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s1 = "hello";  
        String s2 = "HELLO";  
  
        System.out.println(s1.equalsIgnoreCase(s2));  
    }  
}
```

Output?

- A. true
- B. false
- C. Compilation Error
- D. Runtime Error

Q65.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s = null;  
  
        System.out.println(s.length());  
    }  
}
```

Result?

- A. 0
- B. NullPointerException

- C. Compilation Error
- D. Runtime Exception

Q66.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s = "Java";  
  
        System.out.println(s.charAt(4));  
    }  
}
```

Result?

- A. a
- B. NullPointerException
- C. StringIndexOutOfBoundsException
- D. Compilation Error

Q67.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        StringBuilder sb = new StringBuilder("Java");  
  
        sb.append(" Programming");  
  
        System.out.println(sb);  
    }  
}
```

Output?

- A. Java
- B. Java Programming
- C. Compilation Error
- D. Runtime Error

Q68.

Which exception is checked?

- A. ArithmeticException
- B. NullPointerException
- C. IOException
- D. ArrayIndexOutOfBoundsException

Q69.

```
public class Test {  
  
    static void test() throws Exception {  
        throw new Exception("Error");  
    }  
  
    public static void main(String[] args) {  
  
        try {  
            test();  
        }  
        catch(Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output?

- A. Error
- B. Exception
- C. Runtime Error
- D. Compilation Error

Q70.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        try {  
            int x = 10 / 0;  
        }  
  
        finally {  
            System.out.println("Finally");  
        }  
    }  
}
```

Output?

- A. Finally
- B. Runtime Exception only
- C.

Finally
Exception

- D. Compilation Error

Q71.

```
public class Test {  
  
    public static void main(String[] args) {
```

```
        try {
            System.out.println("Try");
            return;
        }

        finally {
            System.out.println("Finally");
        }
    }
}
```

Output?

- A. Try
- B. Finally
- C.

Try
Finally

- D. Compilation Error

Q72.

Which block is ALWAYS executed, whether exception occurs or not?

- A. try
- B. catch
- C. throw
- D. finally

Q73.

```
public class Test {

    static int test() {

        try {
```

```

        return 10;
    }

    finally {
        return 20;
    }
}

public static void main(String[] args) {

    System.out.println(test());
}
}

```

Output?

- A. 10
- B. 20
- C. Compilation Error
- D. Runtime Error

Q74.

```

public class Test {

    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("Java");

        sb.reverse();

        System.out.println(sb);
    }
}

```

Output?

- A. avaJ
- B. Java
- C. Compilation Error
- D. Runtime Error

Q75.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        try {  
  
            String s = null;  
  
            System.out.println(s.length());  
        }  
  
        catch(ArithmeticException e) {  
            System.out.println("Arithmetic");  
        }  
  
        catch(Exception e) {  
            System.out.println("Exception");  
        }  
    }  
}
```

Output?

- A. Arithmetic
- B. Exception
- C. Runtime Error
- D. Compilation Error

Q76.

Which statement about **throw** keyword is correct?

- A. Used to declare an exception
- B. Used to create an exception manually
- C. Used only with checked exceptions
- D. Used to handle exceptions

Q77.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        String s = "Java";  
  
        System.out.println(s.substring(1,3));  
    }  
}
```

Output?

- A. Jav
- B. av
- C. ava
- D. Compilation Error

Q78.

```
class MyException extends Exception {  
  
    MyException(String msg) {  
        super(msg);  
    }  
}  
  
public class Test {
```



```

static void check(int age) throws MyException {

    if(age < 18)
        throw new MyException("Not Eligible");
}

public static void main(String[] args) {

    try {
        check(16);
    }

    catch(MyException e) {
        System.out.println(e.getMessage());
    }
}
}

```

Output?

- A. Not Eligible
- B. MyException
- C. Runtime Error
- D. Compilation Error

Q79.

```

import java.util.StringTokenizer;

public class Test {

    public static void main(String[] args) {

        StringTokenizer st =
            new StringTokenizer("Java Python SQL");

        System.out.println(st.countTokens());
    }
}

```

```
    }  
}
```

Output?

- A. 2
- B. 3
- C. 1
- D. Compilation Error

Q80.

```
public class Test {  
  
    public static void main(String[] args) {  
  
        try {  
            int arr[] = {1,2,3};  
  
            System.out.println(arr[5]);  
        }  
  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Invalid Index");  
        }  
  
        finally {  
            System.out.println("Done");  
        }  
    }  
}
```

Output?

- A.
- Invalid Index
- Done

B.

Done

Invalid Index

C. Runtime Error

D. Compilation Error

Q81.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws
    Exception {

        FileOutputStream fos =
            new FileOutputStream("test.txt");

        fos.write(65);

        fos.close();
    }
}
```

What gets written into the file?

A. 65

B. A

C. Compilation Error

D. Runtime Error

Q82.

Which stream is used for reading character data?

A. FileInputStream

B. BufferedInputStream

- C. FileReader
- D. DataInputStream

Q83.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws
Exception {

        File file = new File("abc.txt");

        System.out.println(file.exists());
    }
}
```

Output?

- A. Always true
- B. Always false
- C. true or false depending on file existence
- D. Compilation Error

Q84.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws
Exception {

        FileInputStream fis =
            new FileInputStream("test.txt");
    }
}
```

```
        System.out.println(fis.read());  
  
        fis.close();  
    }  
}
```

If the file contains character A, output?

- A. A
- B. 65
- C. Compilation Error
- D. Runtime Error

Q85.

Which class provides buffering capability for character streams?

- A. BufferedInputStream
- B. BufferedReader
- C. FileReader
- D. InputStreamReader

Q86.

```
import java.io.*;  
  
public class Test {  
  
    public static void main(String[] args) throws  
Exception {  
  
        BufferedReader br =  
            new BufferedReader(  
                new StringReader("Java"));  
  
        System.out.println(br.readLine());  
    }  
}
```

```
}
```

Output?

- A. Java
- B. null
- C. Compilation Error
- D. Runtime Error

Q87.

Which method is used to close a stream?

- A. stop()
- B. destroy()
- C. terminate()
- D. close()

Q88.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws
Exception {

        FileWriter fw =
            new FileWriter("abc.txt");

        fw.write("Java");

        fw.close();

        FileReader fr =
            new FileReader("abc.txt");
```

```

        int i;

        while((i = fr.read()) != -1) {
            System.out.print((char)i);
        }

        fr.close();
    }
}

```

Output?

- A. Java
- B. ASCII values
- C. Compilation Error
- D. Runtime Error

Q89.

Which stream is best suited for binary data?

- A. FileReader
- B. BufferedReader
- C. FileInputStream
- D. StringTokenizer

Q90.

```

import java.io.*;

public class Test {

    public static void main(String[] args) throws
Exception {

        ByteArrayInputStream bis =
            new ByteArrayInputStream(

```

```
        new byte[]{1,2,3});  
  
        System.out.println(bis.read());  
    }  
}
```

Output?

- A. 0
- B. 1
- C. 2
- D. Compilation Error

Q91.

Which statement about `StringTokenizer` is TRUE?

- A. It belongs to the `java.io` package
- B. It splits the string into tokens
- C. It modifies the original string
- D. It works only with numbers

Q92.

```
import java.util.StringTokenizer;  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        StringTokenizer st =  
            new StringTokenizer(  
                "Java,Python,SQL", ",");  
  
        while(st.hasMoreTokens()) {  
            System.out.print(st.nextToken() + " ");  
        }  
    }  
}
```



```
    }  
}
```

Output?

A.

Java Python SQL

B.

Java,Python,SQL

C. Compilation Error

D. Runtime Error

Q93.

Which interface must be implemented by JDBC drivers?

A. Driver

B. Connection

C. Statement

D. ResultSet

Q94.

```
import java.sql.*;  
  
public class Test {  
  
    public static void main(String[] args) throws  
Exception {  
  
        Connection con = null;  
  
        System.out.println(con);  
    }  
}
```

Output?

- A. Connection Object
- B. null
- C. Compilation Error
- D. SQLException

Q95.

Which method is used to establish a JDBC connection?

- A. DriverManager.getConnection()
- B. Connection.connect()
- C. Driver.connect()
- D. Statement.getConnection()

Q96.

```
import java.sql.*;

public class Test {

    public static void main(String[] args) {

        try {

            Class.forName(
                "com.mysql.cj.jdbc.Driver");

            System.out.println("Driver Loaded");

        } catch (Exception e) {

            System.out.println("Error");

        }

    }

}
```

```
}
```

Purpose of `Class.forName()` here?

- A. Execute SQL query
- B. Load JDBC Driver
- C. Create database
- D. Close connection

Q97.

Which JDBC object is used to execute SQL queries?

- A. ResultSet
- B. Connection
- C. Statement
- D. DriverManager

Q98.

```
import java.sql.*;

public class Test {

    public static void main(String[] args) {

        try {

            Connection con =
                DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/test",
                    "root",
                    "root");

            System.out.println("Connected");

        } catch (SQLException e) {
```

```
        System.out.println("Connection Failed");
    }
}
}
```

If the database connection fails, output?

- A. Connected
- B. SQLException
- C. Connection Failed
- D. Compilation Error

Q99.

Which JDBC component stores the query execution result?

- A. Statement
- B. DriverManager
- C. ResultSet
- D. Connection

Q100.

```
import java.io.*;

public class Test {

    public static void main(String[] args) throws
Exception {

        PrintWriter pw =
            new PrintWriter(System.out);

        pw.println("Java");

        pw.close();
    }
}
```

```
        System.out.println("Programming");  
    }  
}
```

Result?

A.

Java
Programming

B.

Java

C. Runtime Error

D. Compilation Error